

Audit - Exemple - Constats d'audit

Rapport d'audit : Binôme Achraf & Samba (Projet Le Comptoir)

Audité par : Binôme Inès & Amel

Dépôt audité : 23/09/2026

Commit / Tag audité : v1, v2, v3 et v4

Remarque : ce binôme n'a pas réalisé la v5 ; l'audit porte uniquement sur les versions v1 à v4.

Constat 1

- **Localisation :** `Product.java:6`, `CartLine.java:18`, `Cart.java:18-24`, `Receipt.java:7`
- **Gravité :** Majeur
- **Principe violé :** Intégrité des données monétaires (Primitive Obsession)
- **Description :** Tous les montants (prix unitaire, total de ligne, total du panier, total du ticket) sont des `double`. Ce type ne représente pas exactement les décimales ($0,1 + 0,2$ ne donne pas exactement $0,3$), et l'erreur s'accumule à chaque addition. Le `%.2f` du ticket masque l'écart à l'affichage, mais le total stocké peut différer de la somme des lignes arrondies. Les remises en pourcentage et la TVA aggraveront le problème. Il faut passer à `BigDecimal` avec un `RoundingMode` explicite, ou à un type `Money` / des centimes en `long`.

Constat 2

- **Localisation :** `Checkout.java:5` et `Receipt.java:9-12`
- **Gravité :** Majeur
- **Principe violé :** Encapsulation & cohérence des données
- **Description :** `Cart.getLines()` renvoie une vue non modifiable mais vivante de la liste interne, pas une copie. `Checkout` la passe telle quelle à `Receipt`, avec un total figé au même instant. Si le panier change après le paiement, le ticket affiche les nouvelles lignes avec l'ancien total. `Receipt` reçoit aussi le total de l'extérieur, sans pouvoir vérifier qu'il correspond à ses lignes. Il faut copier les lignes (`List.copyOf`) et calculer ou valider le total dans `Receipt`.

Constat 3

- **Localisation :** `Checkout.java:4-6` et `Cart.java:18-24`
- **Gravité :** Majeur
- **Principe violé :** Open/Closed Principle (OCP) & Single Responsibility Principle (SRP)
- **Description :** Le calcul du prix est éparpillé : `Cart` fait la somme brute, `Checkout` ne fait que transmettre, `Receipt` affiche. Rien n'est prévu pour accueillir des remises ou de la TVA, et il n'existe aucune abstraction de règle. La première promotion ira soit dans `Checkout.checkout()`, soit dans `Cart.getTotal()`, et ces méthodes grossiront à chaque nouvelle règle (le schéma des blocs `if / else if` imbriqués). `Cart` n'offre de plus que le total global : ni sous-total, ni comptage sur un sous-ensemble de lignes.

Constat 4

- **Localisation** : `ProductCategory.java:3-5` et `Main.java:6`
- **Gravité** : Majeur
- **Principe violé** : Open/Closed Principle (OCP) & modélisation du domaine
- **Description** : `ProductCategory` est un enum à trois valeurs sans aucune donnée. Il n'a pas de taux de TVA, et `F00DS` regroupe fruits, légumes et tout l'alimentaire (le Kiwi de `Main` est en `F00DS`). Le taux de TVA devra donc être déduit par un `switch` ou un `if` sur l'enum, ailleurs dans le code. Chaque nouvelle catégorie ou chaque changement de taux obligera à modifier tous ces tests.

Constat 5

- **Localisation** : `Receipt.java:17-26`
- **Gravité** : Mineur
- **Principe violé** : Single Responsibility Principle (SRP)
- **Description** : `Receipt` est à la fois un objet de données et un afficheur : `display()` fait le formatage, avec « EUR » écrit en dur. `String.format` est appelé sans `Locale`, donc le séparateur décimal dépend de la machine (1,50 ou 1.50). Le ticket n'a de plus aucune ligne pour une remise, un total HT ou la TVA. Le formatage devrait être extrait dans une classe dédiée avec une `Locale` explicite.

Constat 6

- **Localisation** : `Cart.java:10-12`, `CartLine.java:7-13`, `Product.java:9-14`
- **Gravité** : Mineur
- **Principe violé** : Encapsulation & validation des invariants
- **Description** : `addLine()` n'a pas de test `null` et ne fusionne pas les doublons : deux lignes « Kiwi » restent deux lignes, ce qui faussera les règles du type « au moins 3 articles ». `CartLine` accepte un `product` `null` (l'erreur n'apparaîtra que plus tard, dans `getLineTotal()`). `Product` accepte un prix négatif et des champs `null`. Il manque aussi `removeLine()` et `isEmpty()` dans `Cart`.

Constat 7

- **Localisation** : `Main.java:3`
- **Gravité** : Mineur
- **Principe violé** : Organisation du code
- **Description** : `Main` est dans le package `model`, qui devrait ne contenir que le domaine. Aucun test unitaire n'a été fourni, alors que le calcul du total et les futures remises sont exactement le code à sécuriser.

Audit - Exemple - Point de rupture

Ajout d'une promotion ciblée : Cart ne sait donner qu'un total global, et Checkout n'a aucun endroit où appliquer une remise. Une remise sur un sous-ensemble d'articles oblige à écrire un parcours de `getLines()` dans chaque règle, et le Receipt n'a pas de ligne pour afficher la remise. Un changement touchera donc Cart, Checkout et Receipt en même temps.

Évolution de la fiscalité : ProductCategory ne porte aucun taux, et Cart, CartLine et Receipt ignorent la TVA. Elle devra être ajoutée par un test direct sur l'enum, dans Checkout ou Cart. Ajouter une taxe ou reclasser un produit imposera de modifier chaque endroit qui teste la catégorie, avec des double qui multiplient les écarts d'arrondi.

Audit - Exemple - La 6ème demande client

Demande :

"Pour l'opération 'Mois du Végétal', tout panier contenant au moins trois articles de la catégorie Fruits & Légumes bénéficie d'une remise immédiate de 12 % sur ces articles uniquement. Cette offre ne se cumule toujours pas avec les autres remises : seule la plus avantageuse pour le client s'applique sur la commande."

Pourquoi le code actuel casse :

Il n'existe pas de catégorie « Fruits & Légumes » : tout est FOODS, donc on ne peut pas isoler ces articles. Il faudrait aussi compter et sous-totaliser ce sous-ensemble, ce que Cart ne permet pas. La règle de non-cumul demande de comparer plusieurs remises candidates, et Checkout n'a ni liste de règles ni endroit pour les comparer. Enfin, Receipt n'a pas de ligne de remise à afficher. Il faudrait modifier l'enum, Cart, Checkout et Receipt pour une seule promotion.

Annexe - Pistes d'amélioration

Avec ces changements, la 6ème demande devient une classe de règle à ajouter, sans toucher à Cart ni à Checkout. Product et Receipt passent aussi en BigDecimal (Receipt gagne un champ discount, et son String.format prend Locale.FRANCE).

```
// ProductCategory : la donnée vit dans l'enum, plus de switch ailleurs
public enum ProductCategory {
    FRUITS_VEGETABLES(new BigDecimal("0.055")), // taux donnés à titre d'exemple
    FOODS(new BigDecimal("0.055")),
    DRINKS(new BigDecimal("0.20")),
    OTHERS(new BigDecimal("0.20"));

    private final BigDecimal vatRate;
    ProductCategory(BigDecimal vatRate) { this.vatRate = vatRate; }
    public BigDecimal getVatRate() { return vatRate; }
}

// CartLine : BigDecimal + validation du produit
public CartLine(Product product, int quantity) {
    this.product = Objects.requireNonNull(product, "product");
    if (quantity <= 0) throw new IllegalArgumentException("Quantity must be positive");
    this.quantity = quantity;
}
public BigDecimal getLineTotal() {
    return product.getUnitPrice().multiply(BigDecimal.valueOf(quantity));
}

// Cart : fusion des doublons, sous-totaux par prédicat
public void addLine(CartLine line) {
    Objects.requireNonNull(line, "line");
    for (int i = 0; i < lines.size(); i++) {
        CartLine existing = lines.get(i);
        if (existing.getProduct().getReference().equals(line.getProduct().getReference())) {
            lines.set(i, new CartLine(existing.getProduct(),
                                      existing.getQuantity() + line.getQuantity()));
        }
    }
    lines.add(line);
}
public BigDecimal getTotal() { return getTotal(l -> true); }
public BigDecimal getTotal(Predicate<CartLine> filter) {
    return lines.stream().filter(filter)
        .map(CartLine::getLineTotal)
        .reduce(BigDecimal.ZERO, BigDecimal::add);
}
public int countItems(Predicate<CartLine> filter) {
    return lines.stream().filter(filter).mapToInt(CartLine::getQuantity).sum();
}
```

```
// Une règle = une classe. Ajouter une promo n'oblige plus à modifier Checkout.
public interface DiscountRule {
    BigDecimal discountFor(Cart cart);    // 0 si non applicable
}

public class PlantMonthRule implements DiscountRule {
    public BigDecimal discountFor(Cart cart) {
        Predicate<CartLine> fv =
            l -> l.getProduct().getCategory() == ProductCategory.FRUITES_VEGETABLES;
        if (cart.countItems(fv) < 3) return BigDecimal.ZERO;
        return cart.getTotal(fv).multiply(new BigDecimal("0.12"));
    }
}

// Checkout : non-cumul = on garde la meilleure remise, ticket figé (copie des lignes)
public class Checkout {
    private final List<DiscountRule> rules;
    public Checkout(List<DiscountRule> rules) { this.rules = List.copyOf(rules); }

    public Receipt checkout(Cart cart) {
        BigDecimal best = rules.stream()
            .map(r -> r.discountFor(cart))
            .max(Comparator.naturalOrder())
            .orElse(BigDecimal.ZERO);
        return new Receipt(List.copyOf(cart.getLines()), cart.getTotal(), best);
    }
}
```